Machine Coding: Progress Bar

- **Problem:** Build a reusable `ProgressBar` component: takes a `value` (and optional `max`), animates from 0 to the value, shows the percentage (left / center / right), changes color by progress, and supports an indeterminate mode when the progress is unknown. Show it with static values, a live simulated upload (Start / Pause / Resume / Reset), and a loading bar. It must be accessible, handle bad input (NaN, negative, over 100), and animate smoothly.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Props API, label rules decided
Use cases + architecture + props	5 min	Component contract
Static bar: track + fill + label	8 min	Working bar
Clamp/validate, color, label positions, ARIA	10 min	Correct + accessible
Animation (mount + updates) + indeterminate mode	10 min	Smooth, GPU-friendly
Live demo: simulated upload with timer hook	12 min	Start / Pause / Resume / Reset
Test + wrap-up	10 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"The bar is a presentational component: it gets a number and draws it. I clamp the number, animate with `transform:`*

`scaleX` instead of `width` so the browser doesn't re-layout every frame, and expose it to screen readers with `role="progressbar"` and `aria-valuenow`."

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- `ProgressBar` props: `value`, `max` (default 100), `label`, `labelPosition` (left / center / right), `showValue`, `colorByValue`.
- Fill width = `value / max`, clamped to 0–100 %.
- Animates from 0 on mount and smoothly on every change.
- Color by progress: red < 40 %, amber < 70 %, green otherwise (or one brand color).
- Indeterminate mode when `value` is not provided.
- Demo: 4 static bars (one with `max`), a simulated upload with Start/Pause/Resume/Reset and status text, and a loading bar.

Non-functional requirements

Concern	What it means here
Performance	Animate `transform` (compositor-only), not `width` (layout every frame). One timer for the simulation, cleared on pause/finish/unmount.
Scalability	Pure props-in component, so 100 bars on a page are cheap; no internal timers per bar.
Error handling	NaN / strings / negative / > max / `max <= 0` never break the layout.
Accessibility	`role="progressbar"`, `aria-valuemin/max/now`, `aria-valuetext`, `aria-labelledby` with a visible label; status text in `aria-live`; indeterminate has no `aria-valuenow`; `prefers-reduced-motion`.
Reusability	Same component for uploads, quizzes, storage meters, loading.
Extensibility	Striped / segmented / circular variants, sizes, custom thresholds.

Clarifying questions to ask the interviewer

1. Is the input a percentage, or a value with a max (12 of 20 tasks)?
2. Where should the label go: inside the fill, centered, or outside? What about tiny values (2 %)?
3. Should it animate on mount and on updates? How long?
4. Do we need an indeterminate (unknown progress) state?
5. Color rules: one color, or thresholds (red/amber/green)?
6. Is it read-only display, or should it be interactive (a slider)? (A slider is a different component.)

Assumptions we will build with

- `value` + optional `max`; displayed as a rounded percentage.
- The value label sits over the whole track (dark text), so it's always readable, even at 3 %.
- 0.6 s ease-in-out animation; none with reduced motion.
- `value` undefined → indeterminate sliding bar.

- Thresholds 40 / 70 with an opt-out (`colorByValue={false}`) for brand-colored bars.
- Read-only (not a slider).

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page load	Each static bar animates from 0 to its value
`value={30}`	30 % fill, red, "30%" on the left
`value={10}` label center	10 % fill, "10%" centered and readable
`value={12} max={20}`	60 % fill, amber, "60%"
`value={150}` / `-5` / `NaN` / `max={0}`	100 % / 0 % / 0 % / 0 %, no broken layout
Value changes (live)	Fill glides to the new value
Click Start	Upload starts; status "Uploading… 13%" (announced politely)
Click Pause	Fill stops; "Paused at 13%"; button becomes Resume
Click Resume	Continues from the same value
Reaches 100 %	Stops by itself; "Upload complete"; Start disabled
Click Reset	Back to 0; "Ready to upload"; Reset disabled
No `value`	Indeterminate sliding segment; screen readers hear "Loading"
Reduced motion	No transitions; indeterminate shows a static faded bar
Screen reader	"Course progress, progress bar, 80%"

3. Design the architecture

Module	Responsibility
`components/ProgressBar.jsx`	Presentational bar: validate → percent, animate, label, color, ARIA, indeterminate
`utils/progress.js`	Pure `clamp` , `toPercent(value, max)` , `colorFor(percent)`
`hooks/useSimulatedProgress.js`	Demo data source: interval timer with start/pause/reset, auto-stop at 100
`App`	Demo page: static bars, live upload, indeterminate bar
`App.css`	Track/fill/label styles, transform animation, indeterminate keyframes, reduced motion

```
App
├── STATIC_BARS.map → <ProgressBar key label value max labelPosition />
├── useSimulatedProgress() → { progress, isRunning, isDone, start, pause, reset }
│     └── <ProgressBar label="report.pdf" value={progress} colorByValue={false} />
│         status (aria-live) + Start/Pause/Resume/Reset
```

```
└── <ProgressBar label="Loading data" />            ← indeterminate (no value)
ProgressBar → utils/progress.js (toPercent, colorFor)
```

Start simple. First version: two divs with `width: ${value}%`. Then clamp, ARIA, label positions, the transform animation, indeterminate, and finally the live demo.

4. Design the state / data model

State	Where	Purpose
`shown`	ProgressBar	Percentage currently drawn (lags one frame behind for the mount animation)
`progress`	useSimulatedProgress	Simulated upload value 0–100
`running`	useSimulatedProgress	User intent: started or paused

```
// ProgressBar: everything else is derived from props
const indeterminate = value === undefined || value === null;
const percent = indeterminate ? 0 : toPercent(value, max);   // safe 0..100
const [shown, setShown] = useState(0);
useEffect(() => {
  const frame = requestAnimationFrame(() => setShown(percent)); // next frame → CSS transition runs
  return () => cancelAnimationFrame(frame);
}, [percent]);

// useSimulatedProgress
const [progress, setProgress] = useState(0);
const [running, setRunning] = useState(false);
const active = running && progress < 100;   // derived: finishing stops the timer
```

Senior note

- **The bar has (almost) no state:** it's a function of its props. The only state, `shown`, exists purely for the mount animation.
- **Derived instead of stored:** `percent`, `rounded`, the color, `active`, `isDone` and the status text. There's no `isDone` flag to keep in sync.
- **Ownership:** the *data source* (upload, quiz, API) owns the progress value; `ProgressBar` only displays it. Timers live in the hook, not in the component, so 50 bars don't mean 50 timers.

5. Design the data structures and contracts

Component contract:

```
<ProgressBar
  value={45}               // number; undefined/null → indeterminate
  max={100}                // default 100; value/max → percent
  label="Uploading"        // visible label, also the accessible name (aria-labelledby)
  labelPosition="right"    // "left" | "center" | "right"
```

```
  showValue={true}         // show "45%"
  colorByValue={true}      // thresholds, or one brand color
/>
```

Static demo data (stable `id` used as the React key):

```
{ id: "tasks", label: "Tasks done (12 of 20)", value: 12, max: 20, labelPosition: "right" }
```

Helpers:

```
toPercent(value, max = 100) → 0..100   // NaN, non-numbers, Infinity, max <= 0 → 0
colorFor(percent) → "var(--danger)" | "var(--warning)" | "var(--success)"
useSimulatedProgress(interval = 300) → { progress, isRunning, isDone, start, pause, reset }
```

ARIA output for `value={12} max={20}`:

```
<div role="progressbar" aria-labelledby="…" aria-valuemin="0" aria-valuemax="100"
     aria-valuenow="60" aria-valuetext="60%">
```

6. Trace the data flow

User clicks Start on the upload:

```
Click Start → setRunning(true)
  ↓ active = running && progress < 100 = true → effect starts setInterval(300 ms)
  ↓ tick → setProgress(p => min(100, p + 2..8))
  ↓ App re-renders: <ProgressBar value={progress}> → percent = toPercent(progress)
  ↓ ProgressBar effect: requestAnimationFrame → setShown(percent)
  ↓ style transform: scaleX(percent / 100) → CSS transition glides the fill (compositor only)
  ↓ aria-valuenow / aria-valuetext update; status "Uploading… 37%" (aria-live polite)
… progress reaches 100 → active = false → effect cleanup clearInterval → "Upload complete"
```

7. Important technical challenges

Challenge 1: the mount animation and timer cleanup

- *Problem:* The first version sets the value in a `setTimeout` and returns `() => timer` from the effect. That cleanup *returns* the timer id but never calls `clearTimeout`, so a fast value change or unmount leaves a pending timer that sets state later. A fixed 100 ms delay is also arbitrary.
- *Solution:* Render 0 first, then set the real value on the next animation frame (so the CSS transition has a "from" value), and cancel the frame in the cleanup.

```
const [shown, setShown] = useState(0);
useEffect(() => {
  const frame = requestAnimationFrame(() => setShown(percent));
```

```
  return () => cancelAnimationFrame(frame);   // actually cancels
}, [percent]);
```

Challenge 2: smooth animation without layout work

- *Problem:* Transitioning `width` makes the browser recompute layout and repaint on every frame. With several bars, or a heavy page, the animation stutters, especially on low-end phones.
- *Solution:* Keep the fill full-width and scale it with `transform: scaleX()` from the left edge. Transforms are composited (often on the GPU), and layout isn't touched.

```
.progress-fill { position: absolute; inset: 0; transform-origin: left; transform: scaleX(0);
  transition: transform 0.6s cubic-bezier(0.65, 0, 0.35, 1); }
```

```
<div className="progress-fill" style={{ transform: `scaleX(${shown / 100})` }} />
```

Challenge 3: correct and safe values + readable labels

- *Problem:* `width: ${value}%` with `150` overflows, `-5` or `NaN` gives an invalid style; `12 of 20` needs math; a label *inside* the fill is unreadable (or cut off) at 3–10 %. The first version also put `role: "progressbar"` inside the **style** object, where it does nothing (it must be an attribute).
- *Solution:* One `toPercent` that validates and clamps; the label is an overlay on the whole track; ARIA attributes on the element, not in styles.

```
export function toPercent(value, max = 100) {
  if (!Number.isFinite(value) || !Number.isFinite(max) || max <= 0) return 0;
  return clamp((value / max) * 100, 0, 100);
}
```

Challenge 4: a live progress source with Start / Pause / Resume / Reset

- *Problem:* A `setInterval` that reads `progress` from the closure only sees the first value (stale closure). Forgetting `clearInterval` leaks timers, which speed up after each pause/resume. Stopping at 100 needs an extra "done" flag that can get out of sync.
- *Solution:* A functional update inside the interval, an effect that depends on a *derived* `active` flag, and a cleanup that always clears.

```
const active = running && progress < 100;
useEffect(() => {
  if (!active) return;
  const id = setInterval(() => setProgress((p) => Math.min(100, p + 2 + Math.random() * 6)), interval);
  return () => clearInterval(id);
}, [active, interval]);
```

8. Implement in the correct order

#	Step	Working result
1	Track + fill divs with `width: value%`	Something visible
2	`toPercent` with clamp, NaN/max checks; `max` prop	Safe values
3	Label overlay with left/center/right; rounded %	Readable label
4	`role="progressbar"` + `aria-value*` + visible label via `aria-labelledby`	Accessible
5	`colorFor` thresholds + `colorByValue` opt-out	Color feedback
6	Switch to `transform: scaleX` + transition	Smooth updates
7	Mount animation with `requestAnimationFrame` + cancel in cleanup	Animates from 0
8	Indeterminate mode (no value) with keyframes	Unknown durations
9	`prefers-reduced-motion` styles	Motion-safe
10	`useSimulatedProgress` (interval, functional update, derived `active`)	Live data
11	Start / Pause / Resume / Reset + `aria-live` status	Full demo
12	Node self-check for `toPercent` / `colorFor`	Logic proven

9. Cover all states and edge cases

State / edge case	Handling
Mount	Animates 0 → value
Value update	Glides to the new value
0 % / 100 %	Empty track / full fill
Very small value (1–5 %)	Label still readable (overlay on the track)
Value > max / negative	Clamped to 100 / 0
NaN, Infinity, string	0 (caller must pass numbers)
`max` 0 or negative	0 (no division by zero)
Decimals (37.6)	Fill exact; label and ARIA rounded (38 %)
Indeterminate	Sliding segment; no `aria-valuenow`; `aria-valuetext="Loading"`
Threshold boundaries (40, 70)	Lower bound inclusive (40 → amber, 70 → green)
Upload idle / running / paused / done	"Ready" / "Uploading… n%" / "Paused at n%" / "Upload complete"
Pause then Resume	Continues from the same value; no double timers
Unmount while running	Interval and animation frame cleared
Reduced motion	No transitions; static indeterminate bar
Many bars on a page	No timers per bar; transforms only

10. Final structure

```
_12_progress_bar/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css                    ← global reset
    ├── App.jsx                      ← demo: static bars, live upload, indeterminate
    ├── App.css                      ← tokens, track/fill/label, animation, reduced motion
    ├── utils/
    │   └── progress.js              ← clamp, toPercent, colorFor
    ├── hooks/
    │   └── useSimulatedProgress.js  ← interval-driven fake upload
    └── components/
        └── ProgressBar.jsx          ← reusable accessible bar
```

11. Full working code

src/utils/progress.js

```
export const clamp = (n, min, max) => Math.min(max, Math.max(min, n));

// Any value/max -> a safe 0..100 percentage (NaN, negatives, overflow and max <= 0 handled).
export function toPercent(value, max = 100) {
  if (!Number.isFinite(value) || !Number.isFinite(max) || max <= 0) return 0;
  return clamp((value / max) * 100, 0, 100);
}

// Traffic-light color by progress. Thresholds are inclusive at the lower bound.
export function colorFor(percent) {
  if (percent < 40) return "var(--danger)";
  if (percent < 70) return "var(--warning)";
  return "var(--success)";
}
```

src/components/ProgressBar.jsx

```
import { useEffect, useId, useState } from "react";
import { colorFor, toPercent } from "../utils/progress";

// Reusable, accessible progress bar. Leave `value` undefined for an indeterminate ("unknown") bar.
const ProgressBar = ({ value, max = 100, label, labelPosition = "right", showValue = true, colorByValue
= true }) => {
  const labelId = useId();
  const indeterminate = value === undefined || value === null;
```

```
  const percent = indeterminate ? 0 : toPercent(value, max);
  const rounded = Math.round(percent);

  // Start at 0 and move to the real value on the next frame, so the CSS transition animates on mount
too.
  const [shown, setShown] = useState(0);
  useEffect(() => {
    const frame = requestAnimationFrame(() => setShown(percent));
    return () => cancelAnimationFrame(frame);
  }, [percent]);

  return (
    <div className="progress-block">
      {label && (
        <div className="progress-header">
          <span id={labelId}>{label}</span>
        </div>
      )}
      <div
        className={`progress ${indeterminate ? "indeterminate" : ""}`}
        role="progressbar"
        aria-labelledby={label ? labelId : undefined}
        aria-valuemin={indeterminate ? undefined : 0}
        aria-valuemax={indeterminate ? undefined : 100}
        aria-valuenow={indeterminate ? undefined : rounded}
        aria-valuetext={indeterminate ? "Loading" : `${rounded}%`}
      >
        <div
          className="progress-fill"
          style={
            indeterminate
              ? undefined
              : { transform: `scaleX(${shown / 100})`, background: colorByValue ? colorFor(percent) :
"var(--brand)" }
          }
        />
        {showValue && !indeterminate && (
          <span className={`progress-value ${labelPosition}`} aria-hidden="true">
            {rounded}%
          </span>
        )}
      </div>
    </div>
  );
};

export default ProgressBar;
```

src/hooks/useSimulatedProgress.js

```
import { useEffect, useState } from "react";

// Fake upload: adds a random 2-8% every `interval` ms while running, stops at 100.
export function useSimulatedProgress(interval = 300) {
  const [progress, setProgress] = useState(0);
  const [running, setRunning] = useState(false);
  const active = running && progress < 100; // derived: finishing stops the timer automatically

  useEffect(() => {
    if (!active) return;
    const id = setInterval(() => {
      setProgress((p) => Math.min(100, p + 2 + Math.random() * 6)); // functional update: no stale
progress
    }, interval);
    return () => clearInterval(id); // pause, finish or unmount -> no leaked timer
  }, [active, interval]);

  return {
    progress,
    isRunning: active,
    isDone: progress >= 100,
    start: () => setRunning(true),
    pause: () => setRunning(false),
    reset: () => {
      setRunning(false);
      setProgress(0);
    },
  };
}
```

src/App.jsx

```
import "./App.css";
import ProgressBar from "./components/ProgressBar";
import { useSimulatedProgress } from "./hooks/useSimulatedProgress";

const STATIC_BARS = [
  { id: "storage", label: "Storage used", value: 30, labelPosition: "left" },
  { id: "profile", label: "Profile completed", value: 10, labelPosition: "center" },
  { id: "course", label: "Course progress", value: 80, labelPosition: "right" },
  { id: "tasks", label: "Tasks done (12 of 20)", value: 12, max: 20, labelPosition: "right" },
];

function App() {
  const upload = useSimulatedProgress();
```

```
  const uploadStatus = upload.isDone
    ? "Upload complete"
    : upload.isRunning
      ? `Uploading… ${Math.round(upload.progress)}%`
      : upload.progress > 0
        ? `Paused at ${Math.round(upload.progress)}%`
        : "Ready to upload";

  return (
    <main className="container">
      <h1>Progress Bar</h1>

      <section>
        <h2>Static values</h2>
        {STATIC_BARS.map((bar) => (
          <ProgressBar key={bar.id} label={bar.label} value={bar.value} max={bar.max} labelPosition=
{bar.labelPosition} />
        ))}
      </section>

      <section>
        <h2>Live: simulated upload</h2>
        <ProgressBar label="report.pdf" value={upload.progress} colorByValue={false} />
        <p className="status" aria-live="polite">
          {uploadStatus}
        </p>
        <div className="actions">
          {upload.isRunning ? (
            <button type="button" onClick={upload.pause}>
              Pause
            </button>
          ) : (
            <button type="button" onClick={upload.start} disabled={upload.isDone}>
              {upload.progress > 0 && !upload.isDone ? "Resume" : "Start"}
            </button>
          )}
          <button type="button" className="secondary" onClick={upload.reset} disabled={upload.progress
=== 0}>
            Reset
          </button>
        </div>
      </section>

      <section>
        <h2>Unknown duration</h2>
        <ProgressBar label="Loading data" />
      </section>
```

```
    </main>
  );
}

export default App;
```

`src/App.css`

```
:root {
  --track: #e5e7eb;
  --text: #111827;
  --danger: #f87171;
  --warning: #fbbf24;
  --success: #4ade80;
  --brand: #818cf8;
}

body { background: #f9fafb; color: var(--text); font-family: system-ui, sans-serif; }
.container { max-width: 560px; margin: 32px auto; padding: 0 16px; }
h1 { text-align: center; font-size: 28px; margin-bottom: 8px; }
h2 { font-size: 16px; margin: 24px 0 8px; color: #4b5563; }

.progress-block { margin-bottom: 14px; }
.progress-header { font-size: 14px; font-weight: 600; margin-bottom: 4px; }

.progress { position: relative; height: 22px; background: var(--track); border-radius: 999px; overflow:
hidden; }
.progress-fill {
  position: absolute; inset: 0;
  transform-origin: left;          /* scale from the left edge */
  transform: scaleX(0);
  transition: transform 0.6s cubic-bezier(0.65, 0, 0.35, 1), background-color 0.3s;
}
/* The value sits over the whole track (not inside the fill), so 3% is as readable as 97%. */
.progress-value { position: absolute; inset: 0; display: flex; align-items: center; padding: 0 10px;
font-size: 13px; font-weight: 700; color: var(--text); }
.progress-value.left { justify-content: flex-start; }
.progress-value.center { justify-content: center; }
.progress-value.right { justify-content: flex-end; }

/* Indeterminate: a segment sliding back and forth. */
.progress.indeterminate .progress-fill {
  background: var(--brand);
  transform: none; width: 35%;
  animation: slide 1.2s ease-in-out infinite;
}
@keyframes slide {
```

```
  from { left: -35%; }
  to { left: 100%; }
}

@media (prefers-reduced-motion: reduce) {
  .progress-fill { transition: none; }
  .progress.indeterminate .progress-fill { animation: none; left: 0; width: 100%; opacity: 0.5; }
}

.status { font-size: 14px; color: #4b5563; margin: 4px 0 10px; }
.actions { display: flex; gap: 8px; }
button { padding: 8px 16px; border: none; border-radius: 8px; background: #4f46e5; color: white; font-size: 14px; cursor: pointer; }
button.secondary { background: #e5e7eb; color: var(--text); }
button:disabled { opacity: 0.5; cursor: not-allowed; }
button:focus-visible { outline: 3px solid #a5b4fc; outline-offset: 2px; }
```

`src/index.css`

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: invalid and out-of-range values, `max` scaling, division by zero, readable labels at tiny values, decimals, threshold boundaries, indeterminate mode, mount and update animation, cancelled timers and frames, pause/resume without leaked intervals, auto-stop at 100, reduced motion, accessible names and values.

Trade-offs to mention to the interviewer

- *`transform: scaleX` vs `width`:* transform is smoother (no layout), but scaling distorts child content and rounded corners on the fill, so the label lives outside the fill and the track clips with `overflow: hidden`.
- *Native `<progress>` element vs custom divs:* `<progress>` has built-in semantics and an indeterminate state, but is hard to style consistently across browsers (vendor pseudo-elements). Custom divs + ARIA give full control.
- *Label over the track vs inside the fill:* always readable and simple; "inside the fill" looks nice at high values but needs contrast handling and a fallback outside for small values.
- *Color thresholds:* useful for meters (storage, battery), but misleading for uploads (red at 20 % looks like an error), hence `colorByValue={false}` for the upload.
- *rAF for the mount animation vs CSS `@starting-style`:* rAF works everywhere today; `@starting-style` is pure CSS but newer.

What to add with more time

- Real upload progress via `XMLHttpRequest.upload.onprogress` (or `fetch` with streams), with cancel via `AbortController`.
- Variants: striped/animated, segmented steps, circular (SVG `stroke-dashoffset`), sizes.
- Time remaining / speed estimate (moving average of progress per second).
- Tests: `toPercent` boundaries; RTL for ARIA attributes, Start/Pause/Reset with fake timers, no timer after unmount.
- Storybook stories for every prop combination.

13. Counter questions the interviewer may ask

Requirements and design

Q1. When do you use a determinate vs an indeterminate progress bar?

Determinate when you know how much is done (upload bytes, steps of a wizard). Indeterminate when you don't (waiting for a server). Show a determinate bar whenever possible; it reduces perceived waiting and lets users estimate time.

Q2. Progress bar vs spinner vs skeleton: which and when?

Spinner/indeterminate: short, unknown waits (< 2–3 s). Progress bar: longer operations with measurable progress. Skeleton: loading page content whose layout you know, so the page doesn't jump when data arrives.

Q3. Why not the native `<progress>` element?

It's semantic and supports indeterminate (no `value`), but styling the bar needs browser-specific pseudo-elements (`::-webkit-progress-value`, `::-moz-progress-bar`) and transitions are limited. For a custom design, divs + `role="progressbar"` + ARIA are common. For simple cases, `<progress>` is a great choice.

State design

Q4. Should the progress bar own the progress state?

No. It's presentational: it displays whatever value it gets. The source (upload, quiz, API polling) owns the value. That keeps the bar reusable and testable, and avoids hidden timers inside a display component.

Q5. Why is there a `shown` state at all?

For the mount animation: a CSS transition needs a "before" value. Rendering 0 first and switching to the real value on the next frame makes the browser animate from 0. Without it, the bar would appear already full.

Q6. Why derive `active` instead of setting `running = false` when progress hits 100?

Setting it inside the interval mixes two state updates and is easy to forget. `active = running && progress < 100` makes "finished → stop" automatic: the effect depends on `active`, and its cleanup clears the interval.

Hard parts

Q7. Why is `transform` faster than `width` for animation?

Changing `width` triggers layout → paint → composite every frame. Changing `transform` (and `opacity`) can skip layout and paint, and only re-composite, often on the GPU. That's the difference between 60 fps and dropped frames.

Q8. What's the stale-closure bug with `setInterval` in React?

`setInterval(() => setProgress(progress + 5))` captures `progress` from the render that created the interval, so it keeps setting the same value. Use the functional form `setProgress(p => p + 5)`, or keep the latest value in a ref.

Q9. What was wrong with `return () => timer` in the first version?

The cleanup function must *undo* the effect, but it only returned the timer id and never called `clearTimeout(timer)`. If the value changes quickly or the component unmounts, the old timer still fires and sets stale state.

Q10. Which ARIA attributes does a progress bar need?

`role="progressbar"`, `aria-valuemin`, `aria-valuemax`, `aria-valuenow`, and an accessible name (`aria-label` or `aria-labelledby`). Add `aria-valuetext` for a friendly reading ("60%", "Step 3 of 5"). For indeterminate, omit `aria-valuenow`. Use a polite live region for important milestones, not every percent.

Q11. How do you show real upload progress?

`XMLHttpRequest` has `xhr.upload.onprogress` with `loaded` / `total`; set `value={loaded}` and `max={total}`. With `fetch`, upload progress isn't available directly (download progress is, by reading `response.body` as a stream and counting bytes). Axios exposes `onUploadProgress`.

Implementation steps

Q12. How would you test the component?

Unit-test `toPercent` (boundaries, NaN, max 0) and `colorFor` (39.9 / 40 / 70). RTL: render `value={12} max={20}` and assert `aria-valuenow="60"`; indeterminate has no `aria-valuenow`. With fake timers: Start → advance → value increases; Pause → advance → unchanged; unmount → no warnings, timer cleared.

Q13. How do you avoid announcing every single percent to screen readers?

Don't put the changing number in an assertive live region. The `progressbar` role exposes the value when the user asks for it. Announce milestones ("50% uploaded", "Upload complete") in a polite live region, or throttle the announcements.

Q14. How would you build a circular progress indicator?

An SVG `circle` with `stroke-dasharray = circumference` and `stroke-dashoffset = circumference * (1 - percent / 100)`, rotated -90° to start at the top, with a transition on `stroke-dashoffset`. Same props, same ARIA, different drawing.

Performance and scale

Q15. 200 progress bars update every 100 ms (a file manager). What do you watch for?

Re-renders: memoise each bar (`React.memo`) and pass primitive props so only changed bars render; batch updates; animate with transform; throttle updates to what the eye needs (every 200–500 ms is plenty); virtualise the list if it's long.

Q16. The progress jumps 0 → 90 % instantly, then waits at 90 % for 10 s. How do you make it feel better?

Progress is often not linear. Weight the phases (upload 0–70 %, processing 70–100 %), use an easing "fake" progress that slows down near the end (as NProgress does), and show text for the slow phase ("Processing…"). Never let the bar go backwards.

Q17. How do you estimate time remaining?

Measure speed as a moving average of (Δvalue / Δtime) over the last few seconds; remaining = (max − value) / speed. Smooth it to avoid jumpy numbers, and show "About 2 min left" instead of exact seconds.

Trade-offs and "why not X?"

Q18. Why put the percentage label over the track instead of inside the fill?

Inside the fill, the text can be clipped or unreadable when the fill is small (10 %), and its contrast changes with the fill color. Over the whole track it's always visible with one text color. Inside-the-fill is possible with a fallback outside for small values.

Q19. Why not drive the animation with JavaScript (`setInterval` updating width)?

JS-driven animation runs on the main thread and stutters when the thread is busy; it also re-renders React every frame. CSS transitions on `transform` run on the compositor, are smoother, and need one state change per value.

Q20. Why respect `prefers-reduced-motion`, and what changes?

Some users get dizzy or distracted by motion (vestibular disorders). With reduced motion we jump straight to the value and replace the sliding indeterminate animation with a static bar. The information stays; only the movement goes away.

Q21. Why not a UI-library progress component?

In production, MUI / Chakra / Radix progress components are fine and accessible. In an interview, building it shows you know clamping, ARIA, compositor-friendly animation, effect cleanup and stale closures, which are exactly what they test.